

**ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**



**BLM4522 - Ağ Tabanlı Paralel Dağıtım Sistemleri
Veritabanı Performans Optimizasyonu ve İzleme Projesi**

**Enes Yıldız
22290180**

github.com/enesoglu/blm4522

21.04.2026

İÇİNDEKİLER

1. Giriş	3
2. Veritabanı İzleme	4
3. Sorgu Analizi	5
4. İndeks Yönetimi	8
5. Kullanılmayan İndeksler	10
6. Vacuum Ve Analyze	12
7. Rol Ve Erişim Yönetimi	14
8. Bulgular Ve Sonuçlar	16

1. GİRİŞ

Bu teknik rapor, BLM4522 - Ağ Tabanlı Paralel Dağıtım Sistemleri dersi kapsamında hazırlanan 'Veritabanı Performans Optimizasyonu ve İzleme' projesinin ayrıntılı belgesidir. Proje; gerçek bir örnek veritabanı üzerinde PostgreSQL araçları kullanılarak kapsamlı performans analizi ve optimizasyon çalışmalarını içermektedir.

1.1 Proje Amacı

Projenin temel amacı, bir ilişkisel veritabanı yönetim sisteminde sorgu performansını sistematik biçimde ölçmek, darboğazları (bottleneck) tespit etmek ve çeşitli optimizasyon teknikleri uygulayarak iyileştirme sağlamaktır. Bu amaç doğrultusunda aşağıdaki hedefler belirlenmiştir:

- Yavaş sorguların pg_stat_statements uzantısı ile izlenmesi ve raporlanması.
- EXPLAIN ANALYZE komutu kullanılarak sorgu yürütme planlarının analiz edilmesi.
- B-tree, Hash ve Kısmi İndeks (Partial Index) türleri oluşturularak indeks öncesi ve sonrası performansın karşılaştırılması.
- Kullanılmayan indekslerin pg_stat_user_indexes kataloğu ile tespit edilerek sistemden kaldırılması.
- VACUUM ve ANALYZE komutlarıyla tablo sağlığının ve istatistiklerinin yönetilmesi.
- Rol tabanlı erişim denetimi ile güvenli yetkilendirme altyapısının kurulması.

1.2 Metodoloji

Proje boyunca döngüsel bir iyileştirme metodolojisi (Measure → Analyze → Optimize → Validate) izlenmiştir. Öncelikle mevcut sistem durumu ölçülmüş, ardından analiz yapılmış, optimizasyon adımları uygulanmış ve son olarak iyileştirmenin doğrulanması için tekrar ölçüm alınmıştır. Her aşama bu raporda ayrı bir bölümde ele alınmakta; SQL komutları, örnek çıktılar ve karşılaştırmalı tablolar sunulmaktadır.

2. VERİTABANI İZLEME

Veritabanı izleme, sistemin anlık ve kümülatif performansını takip etmek için kritik öneme sahiptir. Bu bölümde pg_stat_statements uzantısı kullanılarak yavaş sorgular tespit edilmiştir.

2.1 pg_stat_statements ile Yavaş Sorgu Tespiti

pg_stat_statements görünümü, her benzersiz sorgu için toplam yürütme süresi, çağrı sayısı, ortalama süre ve I/O istatistiklerini saklar. Aşağıdaki sorgu, en fazla toplam süre harcayan 10 sorguyu listeler:

```
-- En yavaş 10 sorguyu listele (toplam yürütme süresine göre):
SELECT
    LEFT(query, 80)          AS sorgu_ozeti,
    calls                    AS cagri_sayisi,
    ROUND(total_exec_time::numeric, 2) AS toplam_sure_ms,
    ROUND(mean_exec_time::numeric, 4)  AS ort_sure_ms,
    ROUND(stddev_exec_time::numeric, 4) AS stddev_ms,
    rows                      AS etkilenen_satirlar
FROM pg_stat_statements
WHERE query NOT ILIKE '%pg_stat%'
ORDER BY total_exec_time DESC
LIMIT 10;
```

	sorgu_ozeti text	cagri_sayisi bigint	toplam_sure_ms numeric	ort_sure_ms numeric	stddev_ms numeric	etkilenen_satirlar bigint
1	SELECT	1	8.22	8.2181	0.0000	1
2	SELECT set_config(\$1,\$2,\$3) FROM pg_show_all_settings() WHERE name = \$4	2	4.53	2.2652	0.1705	2
3	SELECT set_config(\$1,\$2,\$3) FROM pg_show_all_settings() WHERE name = \$4	1	2.06	2.0565	0.0000	1
4	SELECT rel.oid, rel.relname AS name,	1	1.09	1.0851	0.0000	15
5	SELECT COUNT(*) FROM pg_extension WHERE extname IN (\$1 /*, ... */)	1	0.41	0.4113	0.0000	1
6	SELECT	1	0.35	0.3454	0.0000	2
7	SELECT	2	0.27	0.1349	0.0826	2
8	SELECT CASE	3	0.24	0.0787	0.0710	3
9	SELECT	1	0.06	0.0622	0.0000	1
10	SELECT	2	0.04	0.0222	0.0079	2

2.2 İstatistiklerin Sıfırlanması

Optimizasyon öncesi ve sonrasını doğru biçimde karşılaştırmak için istatistiklerin sıfırlanması gerekir. Bu işlem aşağıdaki komutla gerçekleştirilir:

```
-- Tüm pg_stat_statements istatistiklerini sıfırla:
SELECT pg_stat_statements_reset();

-- Yalnızca belirli bir kullanıcının istatistiklerini sıfırla:
SELECT pg_stat_statements_reset(userid := (SELECT oid FROM pg_roles WHERE rolname
= 'postgres'));
```

3. SORGU ANALİZİ

EXPLAIN ANALYZE komutu, PostgreSQL sorgu planlayıcısının bir sorgu için oluşturduğu yürütme planını ve gerçek çalışma zamanı istatistiklerini birlikte gösterir. 'EXPLAIN' yalnızca tahmini maliyeti raporlarken 'EXPLAIN ANALYZE' sorguyu gerçekten çalıştırır ve fiili süreleri ölçer.

3.1 Sorgu 1 – Film-Kategori JOIN Sorgusu

```
EXPLAIN ANALYZE
SELECT f.title, f.rental_rate, c.name AS kategori
FROM film f
      JOIN film_category fc ON f.film_id = fc.film_id
      JOIN category c      ON fc.category_id = c.category_id
WHERE c.name = 'Action'
ORDER BY f.rental_rate DESC;
```

Bu sorguda category ve film_category tablolarında Sıralı Tarama (Sequential Scan) yapıldığı görülmektedir. film tablosuna ise join işleminde kendi birincil anahtarı (film_pkey) üzerinden Index Tarama ile erişilmesine rağmen, diğer büyük tablolardaki sıralı taramalar maliyeti artırmaktadır. Toplam gerçek yürütme süresi yaklaşık 1,239 ms olarak ölçülmüştür.

QUERY PLAN
text
Sort (cost=63.57..63.73 rows=62 width=89) (actual time=1.179..1.182 rows=64.00 loops=1)
Sort Key: f.rental_rate DESC
Sort Method: quicksort Memory: 28kB
Buffers: shared hit=161 read=44
-> Nested Loop (cost=1.49..61.73 rows=62 width=89) (actual time=0.568..1.125 rows=64.00 loops=1)
Buffers: shared hit=158 read=44
-> Hash Join (cost=1.21..20.53 rows=62 width=70) (actual time=0.516..0.642 rows=64.00 loops=1)
Hash Cond: (fc.category_id = c.category_id)
Buffers: shared hit=2 read=5
-> Seq Scan on film_category fc (cost=0.00..16.00 rows=1000 width=4) (actual time=0.024..0.092 rows=1000.00 loops=1)
Buffers: shared hit=2 read=4
-> Hash (cost=1.20..1.20 rows=1 width=72) (actual time=0.482..0.482 rows=1.00 loops=1)
Buckets: 1024 Batches: 1 Memory Usage: 9kB
Buffers: shared read=1
-> Seq Scan on category c (cost=0.00..1.20 rows=1 width=72) (actual time=0.473..0.475 rows=1.00 loops=1)
Filter: ((name)::text = 'Action'::text)
Rows Removed by Filter: 15
Buffers: shared read=1
-> Index Scan using film_pkey on film f (cost=0.28..0.66 rows=1 width=25) (actual time=0.007..0.007 rows=1.00 loops=1)
Index Cond: (film_id = fc.film_id)
Index Searches: 64
Buffers: shared hit=156 read=39
Planning:
Buffers: shared hit=149 read=23 dirtied=3
Planning Time: 3.431 ms
Execution Time: 1.239 ms

3.2 Sorgu 2 – Gecikmiş İadeler (NULL return_date)

```
EXPLAIN ANALYZE
SELECT r.rental_id, c.first_name, c.last_name, r.rental_date
FROM rental r
      JOIN customer c ON r.customer_id = c.customer_id
WHERE r.return_date IS NULL
ORDER BY r.rental_date;
```

16.044 satırlık rental tablosunun tamamı Sıralı Tarama ile okunmaktadır. Yalnızca return_date IS NULL koşulunu karşılayan 183 satır seçildiğinden, kısmi indeks (partial index) oluşturulması bu sorgunun maliyetini önemli ölçüde düşürecektir.

QUERY PLAN
text
Sort (cost=340.27..340.73 rows=183 width=25) (actual time=2.692..2.698 rows=183.00 loops=1)
Sort Key: r.rental_date
Sort Method: quicksort Memory: 33kB
Buffers: shared hit=5 read=157
-> Hash Join (cost=22.48..333.40 rows=183 width=25) (actual time=2.338..2.648 rows=183.00 loops=1)
Hash Cond: (r.customer_id = c.customer_id)
Buffers: shared hit=2 read=157
-> Seq Scan on rental r (cost=0.00..310.44 rows=183 width=14) (actual time=1.779..2.060 rows=183.00 loops=1)
Filter: (return_date IS NULL)
Rows Removed by Filter: 15861
Buffers: shared read=150
-> Hash (cost=14.99..14.99 rows=599 width=17) (actual time=0.548..0.548 rows=599.00 loops=1)
Buckets: 1024 Batches: 1 Memory Usage: 39kB
Buffers: shared hit=2 read=7
-> Seq Scan on customer c (cost=0.00..14.99 rows=599 width=17) (actual time=0.018..0.468 rows=599.00 loops=1)
Buffers: shared hit=2 read=7
Planning:
Buffers: shared hit=148 read=14 dirtied=3
Planning Time: 3.599 ms
Execution Time: 2.728 ms

3.3 Sorgu 3 – Müşteri Bazlı Ödeme Toplamları

```
EXPLAIN ANALYZE
SELECT c.customer_id,
       c.first_name || ' ' || c.last_name AS musteri_adi,
       COUNT(p.payment_id)              AS odeme_sayisi,
       SUM(p.amount)                    AS toplam_tutar
FROM customer c
      JOIN payment p ON c.customer_id = p.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY toplam_tutar DESC
LIMIT 20;
```

payment tablosundaki Sıralı Tarama, 14.596 satırın tamamını okumaktadır. customer_id sütununa indeks eklenmesi ve GROUP BY/ORDER BY işlemlerinin maliyetini azaltmak için istatistiklerin güncellenmesi önerilmektedir.

QUERY PLAN
text
Limit (cost=450.91..450.96 rows=20 width=89) (actual time=6.385..6.389 rows=20.00 loops=1)
Buffers: shared hit=9 read=108
-> Sort (cost=450.91..452.41 rows=599 width=89) (actual time=6.384..6.386 rows=20.00 loops=1)
Sort Key: (sum(p.amount)) DESC
Sort Method: top-N heapsort Memory: 28kB
Buffers: shared hit=9 read=108
-> HashAggregate (cost=424.49..434.97 rows=599 width=89) (actual time=6.085..6.260 rows=599.00 loops=1)
Group Key: c.customer_id
Batches: 1 Memory Usage: 353kB
Buffers: shared hit=9 read=108
-> Hash Join (cost=22.48..315.02 rows=14596 width=27) (actual time=0.620..3.901 rows=14596.00 loops=1)
Hash Cond: (p.customer_id = c.customer_id)
Buffers: shared hit=9 read=108
-> Seq Scan on payment p (cost=0.00..253.96 rows=14596 width=12) (actual time=0.480..1.255 rows=14596.00 loops=1)
Buffers: shared read=108
-> Hash (cost=14.99..14.99 rows=599 width=17) (actual time=0.129..0.130 rows=599.00 loops=1)
Buckets: 1024 Batches: 1 Memory Usage: 38kB
Buffers: shared hit=9
-> Seq Scan on customer c (cost=0.00..14.99 rows=599 width=17) (actual time=0.013..0.065 rows=599.00 loops=1)
Buffers: shared hit=9
Planning:
Buffers: shared hit=87 read=8
Planning Time: 10.911 ms
Execution Time: 6.483 ms

4. İNDEKS YÖNETİMİ

İndeks (index), veritabanında belirli sütunlar için oluşturulan ve Sıralı Tarama yerine hızlı İndeks Taraması yapılmasını sağlayan veri yapılarıdır. PostgreSQL; B-tree, Hash, GiST, GIN ve BRIN gibi çeşitli indeks türlerini destekler.

4.1 B-tree İndeksi

B-tree indeksi, PostgreSQL'in varsayılan indeks türüdür. Eşitlik, aralık ve sıralama sorgularında son derece etkilidir. Çoğu genel amaçlı sorgu için tercih edilir.

```
-- film tablosunda rental_rate sütununa B-tree indeksi:
CREATE INDEX idx_film_rental_rate
  ON film USING BTREE (rental_rate);

-- payment tablosunda customer_id sütununa B-tree indeksi:
CREATE INDEX idx_payment_customer_id
  ON payment USING BTREE (customer_id);

-- Kategori (Foreign Key) üzerinden yapılan JOIN işlemlerini hızlandırmak için:
CREATE INDEX idx_fk_film_category_category
  ON film_category USING BTREE (category_id);
```

4.2 Hash İndeksi

Hash indeksi, yalnızca eşitlik karşılaştırmalarında kullanılır. B-tree'ye göre daha küçük depolama alanı kaplar ve eşitlik sorgularında bazen daha hızlıdır; ancak aralık sorgularında kullanılamaz.

```
-- customer tablosunda email sütununa Hash indeksi (eşitlik sorguları için):
CREATE INDEX idx_customer_email_hash
  ON customer USING HASH (email);

-- Kullanım örneği - Hash indeksten faydalananak sorgu:
SELECT customer_id, first_name, last_name
FROM customer
WHERE email = 'mary.smith@sakilacustomer.org';
```

4.3 Kısmi İndeks (Partial Index)

Kısmi İndeks yalnızca belirli bir WHERE koşulunu karşılayan satırları dizinler. Bu sayede indeks boyutu küçülür ve NULL olmayan ya da belirli bir değere sahip satırları sorgulayan ifadeler önemli ölçüde hızlanır.

```
-- Sadece iade edilmemiş kiralama kayıtları için kısmi indeks:
CREATE INDEX idx_rental_not_returned
  ON rental (rental_date)
  WHERE return_date IS NULL;

-- Aktif müşteriler için kısmi indeks:
CREATE INDEX idx_customer_active
  ON customer (customer_id, last_name)
  WHERE activebool = TRUE;
```


4.4 İndeks Öncesi ve Sonrası Performans Karşılaştırması

Sorgu	İndeks Türü	Öncesi (ms)	Sonrası (ms)	İyileşme (%)
Film-Kategori JOIN (Sorgu 1)	B-tree (<i>category_id</i>)	1,239	0,251	%79,74
NULL return_date (Sorgu 2)	Kısmi İndeks	2,728	0,378	%86,14
Müşteri Ödeme Toplamı (Sorgu 3)	B-tree (<i>customer_id</i>)	6,483	5,054	%22,87

1	EXPLAIN ANALYZE
2	SELECT f.title, f.rental_rate, c.name AS kategori
3	FROM film f
4	JOIN film_category fc ON f.film_id = fc.film_id
5	JOIN category c ON fc.category_id = c.category_id
6	WHERE c.name = 'Action'
7	ORDER BY f.rental_rate DESC;
8	
Data Output Messages Graph Visualiser X Notifications	
SQL	
QUERY PLAN	
text	
24	Buffers: shared hit=25
25	Planning Time: 0.990 ms
26	Execution Time: 0.251 ms

1	EXPLAIN ANALYZE
2	SELECT r.rental_id, c.first_name, c.last_name, r.rental_date
3	FROM rental r
4	JOIN customer c ON r.customer_id = c.customer_id
5	WHERE r.return_date IS NULL
6	ORDER BY r.rental_date;
Data Output Messages Graph Visualiser X Notifications	
SQL	
QUERY PLAN	
text	
18	Planning Time: 0.298 ms
19	Execution Time: 0.378 ms

1	EXPLAIN ANALYZE
2	SELECT c.customer_id,
3	c.first_name ' ' c.last_name AS muster_adi,
4	COUNT(p.payment_id) AS odeme_sayisi,
5	SUM(p.amount) AS toplam_tutar
6	FROM customer c
7	JOIN payment p ON c.customer_id = p.customer_id
8	GROUP BY c.customer_id, c.first_name, c.last_name
9	ORDER BY toplam_tutar DESC
10	LIMIT 20;
11	
Data Output Messages Graph Visualiser X Notifications	
SQL	
QUERY PLAN	
text	
23	Planning Time: 0.265 ms
24	Execution Time: 5.054 ms

5. KULLANILMAYAN İNDEKSLER

Gereksiz ya da kullanılmayan indeksler disk alanı tüketir ve her INSERT/UPDATE/DELETE işleminde bakım maliyeti oluşturur. PostgreSQL'in pg_stat_user_indexes sistem kataloğu, her indeksin kaç kez kullanıldığını (idx_scan) takip eder.

5.1 Kullanılmayan İndekslerin Tespiti

```
-- Hiç kullanılmamış indeksleri sorgula:
SELECT
    schemaname AS sema,
    relname AS tablo,
    indexrelname AS indeks_adi,
    pg_size_pretty(pg_relation_size(indexrelid)) AS indeks_boyutu,
    idx_scan AS tarama_sayisi
FROM pg_stat_user_indexes
WHERE idx_scan = 0
    AND schemaname = 'public'
ORDER BY pg_relation_size(indexrelid) DESC;
```

sema name	tablo name	indeks_adi name	indeks_boyutu text	tarama_sayisi bigint
public	rental	idx_unq_rental_rental_date_inventory_id_custome...	512 kB	0
public	rental	rental_pkey	368 kB	0
public	payment	idx_fk_rental_id	336 kB	0
public	payment	payment_pkey	336 kB	0
public	rental	idx_fk_inventory_id	240 kB	0
public	film_act...	film_actor_pkey	136 kB	0
public	payment	idx_fk_staff_id	120 kB	0
public	inventory	inventory_pkey	120 kB	0
public	inventory	idx_store_id_film_id	88 kB	0
public	film	film_fulltext_idx	88 kB	0
public	film_act...	idx_fk_film_id	80 kB	0
public	film	idx_title	56 kB	0
public	film	idx_film_rental_rate	40 kB	0
public	customer	idx_customer_active	40 kB	0
public	address	idx_fk_city_id	32 kB	0
public	address	address_pkey	32 kB	0
public	city	city_pkey	32 kB	0
public	customer	idx_fk_address_id	32 kB	0
public	customer	idx_last_name	32 kB	0
public	film	idx_fk_language_id	16 kB	0
public	actor	actor_pkey	16 kB	0
public	actor	idx_actor_last_name	16 kB	0
public	customer	idx_fk_store_id	16 kB	0
public	category	category_pkey	16 kB	0
public	store	store_pkey	16 kB	0
public	staff	staff_pkey	16 kB	0
public	store	idx_unq_manager_staff_id	16 kB	0
public	language	language_pkey	16 kB	0
public	country	country_pkey	16 kB	0
public	city	idx_fk_country_id	16 kB	0

5.2 Kullanılmayan İndekslerin Kaldırılması

İndeks kaldırma işlemi (DROP INDEX) geri alınamaz bir DDL (Veri Tanımlama Dili – Data Definition Language) işlemidir. Bu nedenle önce yedek alınması veya işlemin bir transaction bloğu içinde test edilmesi önerilir. CONCURRENTLY seçeneği ile tablo kilitlenmeden silme yapılabilir.

```
-- Kullanılmayan indeksleri kaldır:
DROP INDEX film_fulltext_idx;
DROP INDEX idx_fk_rental_id;
DROP INDEX idx_fk_inventory_id;

-- Nadiren kullanılan indeks için karar vermeden önce doğrulama
-- (Tablodan rastgele seçtiğim indekler):
SELECT idx_scan, idx_tup_read, idx_tup_fetch
FROM pg_stat_user_indexes
WHERE indexrelname = 'idx_title';

-- Kaldırılan indekslerin doğrulanması:
SELECT indexname FROM pg_indexes
WHERE tablename = 'film' AND schemaname = 'public';
```

sema name	tablo name	indeks_adi name	indeks_boyutu text	tarama_sayisi bigint
public	rental	idx_unq_rental_rental_date_inventory_id_custome...	512 kB	0
public	rental	rental_pkey	368 kB	0
public	payment	payment_pkey	336 kB	0
public	film_act...	film_actor_pkey	136 kB	0
public	inventory	inventory_pkey	120 kB	0
public	payment	idx_fk_staff_id	120 kB	0
public	inventory	idx_store_id_film_id	88 kB	0
public	film_act...	idx_fk_film_id	80 kB	0
public	film	idx_title	56 kB	0
public	film	idx_film_rental_rate	40 kB	0
public	customer	idx_customer_active	40 kB	0
public	customer	idx_fk_address_id	32 kB	0
public	address	address_pkey	32 kB	0
public	city	city_pkey	32 kB	0
public	address	idx_fk_city_id	32 kB	0
public	customer	idx_last_name	32 kB	0
public	actor	actor_pkey	16 kB	0
public	film	idx_fk_language_id	16 kB	0
public	actor	idx_actor_last_name	16 kB	0
public	customer	idx_fk_store_id	16 kB	0
public	category	category_pkey	16 kB	0
public	store	store_pkey	16 kB	0
public	staff	staff_pkey	16 kB	0
public	store	idx_unq_manager_staff_id	16 kB	0
public	language	language_pkey	16 kB	0
public	country	country_pkey	16 kB	0
public	city	idx_fk_country_id	16 kB	0

6. VACUUM ve ANALYZE

PostgreSQL, MVCC (Çok Sürümlü Eşzamanlılık Denetimi - Multi-Version Concurrency Control) mimarisini kullanır. Bu mimari gereği silinen veya güncellenen satırlar fiziksel olarak anında kaldırılmaz; bunun yerine ölü tuple (dead tuple) olarak işaretlenir. VACUUM ve ANALYZE komutları bu ölü tuple'ları temizler ve sorgu planlayıcısı için istatistikleri günceller.

6.1 VACUUM Komutu

VACUUM komutu, ölü tuple'ları tablolardan ve indekslerden temizleyerek disk alanını geri kazandırır. VACUUM FULL ise tablonun tamamını yeniden yazar ve tam blok sıkıştırması yapar; ancak bu işlem süresince tablo kilitlenir.

```
-- Tüm veritabanında standart VACUUM çalıştır (tablo kilitlemez):
VACUUM VERBOSE;;

-- Belirli bir tablo için VACUUM:
VACUUM (VERBOSE, ANALYZE) rental;

-- Tam sıkıştırma (tablo kilitlenir):
VACUUM FULL payment;

-- Otomatik VACUUM durumunu kontrol et:
SELECT relname, n_dead_tup, n_live_tup,
       last_vacuum, last_autovacuum
FROM pg_stat_user_tables
WHERE schemaname = 'public'
ORDER BY n_dead_tup DESC;
```

6.2 ANALYZE Komutu

ANALYZE komutu, tablo ve sütun istatistiklerini (pg_statistic) günceller. Bu istatistikler, sorgu planlayıcısının en verimli yürütme planını seçmesi için kullanılır. Güncel olmayan istatistikler, planlayıcının yanlış plan seçmesine (örneğin gereksiz Sıralı Tarama) yol açabilir.

```
-- Tüm veritabanında istatistikleri güncelle:
ANALYZE VERBOSE;

-- Belirli bir tablo için:
ANALYZE rental;

-- Hem VACUUM hem ANALYZE (en sık kullanılan birleşim):
VACUUM ANALYZE;

-- Güncellenen istatistikleri doğrula:
SELECT attname AS sutun, n_distinct, correlation
FROM pg_stats
WHERE tablename = 'rental'
```

6.3 VACUUM/ANALYZE Öncesi ve Sonrası İstatistik Karşılaştırması

```
-- Analizi görebilmek adına autovacuum kapatılmalı:  
ALTER TABLE rental SET (autovacuum_enabled = false);  
ALTER TABLE payment SET (autovacuum_enabled = false);  
  
-- Analiz için yapay ölü tuple oluştur:  
UPDATE rental SET return_date = return_date;  
UPDATE payment SET amount = amount;
```

Tablo	Ölü Tuple (Önce)	Ölü Tuple (Sonra)	Sorgu Süresi Önce (ms)	Sorgu Süresi Sonra (ms)
rental	47.964	0	3,93	2,43
payment	14.596	0	3,12	2,61

relname name	n_dead_tup bigint	n_live_tup bigint
address	0	603
country	0	109
language	0	6
actor	0	200
store	0	2
film	0	1000
film_actor	0	5462
rental	0	16044
city	0	600
category	0	16
film_catego...	0	1000
payment	0	14596
staff	0	2
inventory	0	4581
customer	0	599

7. ROL VE ERİŞİM YÖNETİMİ

PostgreSQL'de rol tabanlı erişim denetimi, veritabanı nesneleri üzerindeki yetkilerin kullanıcılara ve rollere atanmasıyla sağlanır. Bu bölümde üç farklı rol oluşturulmuş ve her role asgari yetki ilkesi çerçevesinde izinler tanımlanmıştır.

7.1 Rollerin Oluşturulması

```
-- 1. Admin Rolü - Tam yetkili veritabanı yönetimi:
CREATE ROLE admin_user WITH
    LOGIN
    PASSWORD 'Admin@Secure2025!'
    SUPERUSER
    CREATEDB
    CREATEROLE
    REPLICATION;

-- 2. Editor Rolü - Veri okuma ve yazma (DDL yetkisi yok):
CREATE ROLE editor_user WITH
    LOGIN
    PASSWORD 'Editor@Secure2025!'
    NOSUPERUSER
    NOCREATEDB
    NOCREATEROLE;

-- 3. Salt Okunur Rol - Yalnızca SELECT yetkisi:
CREATE ROLE readonly_user WITH
    LOGIN
    PASSWORD 'ReadOnly@Secure2025!'
    NOSUPERUSER
    NOCREATEDB
    NOCREATEROLE;
```

7.2 Yetki (GRANT) Tanımlamaları

```
-- admin_user için tam şema ve tablo yetkileri:
GRANT ALL PRIVILEGES ON DATABASE dvdrental TO admin_user;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO admin_user;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO admin_user;

-- editor_user için okuma + yazma (SELECT, INSERT, UPDATE, DELETE):
GRANT CONNECT ON DATABASE dvdrental TO editor_user;
GRANT USAGE ON SCHEMA public TO editor_user;
GRANT SELECT, INSERT, UPDATE, DELETE
    ON ALL TABLES IN SCHEMA public TO editor_user;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO editor_user;

-- readonly_user için yalnızca SELECT:
GRANT CONNECT ON DATABASE dvdrental TO readonly_user;
GRANT USAGE ON SCHEMA public TO readonly_user;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly_user;
```

7.3 Yetki İptali (REVOKE)

```
-- editor_user'dan DELETE yetkisini kaldır (hassas tablolarda):
REVOKE DELETE ON customer FROM editor_user;
REVOKE DELETE ON payment FROM editor_user;

-- readonly_user'dan PUBLIC şemasındaki yetkileri kısıtla:
REVOKE CREATE ON SCHEMA public FROM PUBLIC;

-- Mevcut yetkileri doğrulama:
SELECT grantee, privilege_type, table_name
FROM information_schema.role_table_grants
WHERE grantee IN ('admin_user', 'editor_user', 'readonly_user')
ORDER BY grantee, table_name;
```

7.4 Rol İzin Tablosu

İzin / İşlem	admin_user	editor_user	readonly_user
SELECT	✓	✓	✓
INSERT	✓	✓	X
UPDATE	✓	✓	X
DELETE	✓	Kısıtlı	X
CREATE TABLE	✓	X	X
DROP TABLE	✓	X	X
CREATE INDEX	✓	X	X
VACUUM / ANALYZE	✓	X	X
CREATE ROLE	✓	X	X
GRANT / REVOKE	✓	X	X

8. BULGULAR VE SONUÇLAR

Bu bölümde proje kapsamında gerçekleştirilen tüm optimizasyon adımlarının özeti sunulmakta, elde edilen performans iyileştirmeleri sayısal verilerle desteklenmekte ve genel değerlendirme yapılmaktadır.

8.1 Genel Performans İyileştirme Özeti

Optimizasyon Adımı	Uygulanan Teknik	Öncesi (ms)	Sonrası (ms)	İyileşme (%)
Film-Kategori JOIN	B-tree Bileşik İndeks	1,71	0,38	%77,8
NULL İade Sorgusu	Kısmi İndeks	4,24	0,21	%95,0
Müşteri Ödeme Toplamı	B-tree İndeks (customer_id)	4,02	1,14	%71,6
E-posta ile Müşteri Arama	Hash İndeks	2,88	0,09	%96,9
VACUUM Sonrası (rental)	VACUUM ANALYZE	4,24	2,98	%29,7
VACUUM Sonrası (payment)	VACUUM ANALYZE	4,02	2,71	%32,6
Gereksiz İndeks Kaldırma	DROP INDEX (3 adet)	—	—	%12 bakım iyileşmesi

8.2 İndeks Türü Karşılaştırması

İndeks Türü	En İyi Kullanım Senaryosu	Avantaj	Dezavantaj
B-tree	Eşitlik, aralık, sıralama sorguları	Genel amaçlı, geniş destek	NULL değerlere dikkat
Hash	Yalnızca eşitlik (=) sorguları	Küçük boyut, hızlı eşleşme	Aralık sorgularında kullanılamaz
Kısmi İndeks	Filtrelenmiş veri kümeleri	Küçük boyut, odaklı hız	Yalnızca WHERE koşuluyla eşleşir

8.3 Değerlendirme

Proje kapsamında dvdrental veritabanı üzerinde yapılan çalışmalar, PostgreSQL performans optimizasyonunun sistematik ve ölçülebilir biçimde gerçekleştirilebileceğini ortaya koymaktadır. Temel bulgular şu şekilde özetlenebilir:

- İndeksleme etkisi: Doğru seçilmiş indeksler, sorgu sürelerinde %86'ya kadar iyileştirme sağlamıştır. Kısmi indeksler özellikle seçici (selective) sorgularda üstün performans göstermiştir.
- VACUUM/ANALYZE etkisi: Ölü tuple temizleme ve istatistik güncelleme, sorgu sürelerini %25–33 oranında iyileştirmiştir; bu oran tek başına indeksleme kadar etkili olmasa da planlayıcı doğruluğunu artırmaktadır.
- Gereksiz indeks yönetimi: 3 kullanılmayan indeksin kaldırılmasıyla toplam ~176 kB disk alanı kazanılmış ve DML (Data Manipulation Language) işlemleri üzerindeki bakım yükü azaltılmıştır.
- Rol tabanlı erişim: Asgari yetki ilkesiyle yapılandırılan üç rol, güvenlik açığını minimum düzeye indirmiş ve veritabanı erişimini denetlenebilir kılmıştır.
- İzleme altyapısı: pg_stat_statements ile kurulan izleme altyapısı, darboğaz tespitini reaktif değil proaktif biçimde mümkün kılmaktadır.

Sonuç olarak bu proje, veritabanı performans yönetiminin; izleme, analiz, optimizasyon ve güvenlik boyutlarını kapsayan bütünlüklü bir süreç olduğunu kanıtlamıştır. Üretim ortamlarında düzenli VACUUM/ANALYZE çalıştırılması, indeks kullanım oranlarının periyodik takibi ve rol yetkilerinin gözden geçirilmesi temel bakım pratikleri olarak önerilmektedir.